

Task 1: The ElGamal cryptosystem

```
ElGamal Cryptosystem - Comprehensive Test Suite
=====

Generating keys (256-bit)...
✓ Keys generated successfully
  - Modulus p: 256 bits
  - Generator g: 73944594973202724538891717281904032798034293274239571608665201638040070099538

Test 1: Basic Encryption and Decryption
✓ PASSED - Message: 12345, Decrypted: 12345

Test 2: Homomorphic Multiplication (Small Numbers)
✓ PASSED - 5 × 6 = 30

Test 3: Homomorphic Multiplication (Large Random Numbers)
✓ PASSED - 4374 × 1442 = 6307308

Test 4: Semantic Security (Probabilistic Encryption)
✓ PASSED - Same message produces different ciphertexts
  CT1: 3602...1035
  CT2: 9301...8128

Test 5: Multiplication by Identity Element (1)
✓ PASSED - 456 × 1 = 456

Test 6: Associativity of Multiplication
✓ PASSED - (10×20)×30 = 10×(20×30) = 6000

Test 7: Commutativity of Multiplication
✓ PASSED - 50×4 = 4×50 = 200

Test 8: Boundary Condition (Maximum Message)
✓ PASSED - Max message (p-1) encrypted and decrypted correctly

Test 9: Error Handling (Message > p)
✓ PASSED - ValueError raised as expected
  Error message: Message is too large for the modulus p.

Test 10: Security Integrity (Wrong Private Key)
✓ PASSED - Wrong key produces incorrect decryption
  Original: 100, With wrong key: 88887223113201249056209120120507207614711233925155156410760814236763097244641

=====
TEST RESULTS: 10/10 PASSED
=====
ALL TESTS PASSED SUCCESSFULLY!
=====
```

1. Rationale for Test Case Selection

We selected 10 test cases to verify the correctness, mathematical integrity, and security of our ElGamal implementation.

- **Test 1 (Basic Encryption/Decryption):** We tested the fundamental cycle of encrypting and decrypting a standard integer to ensure the core algorithm functions correctly.
- **Test 2 (Small Homomorphic Multiplication):** We verified the multiplication property with small numbers (5 and 6) to easily confirm that operations on ciphertexts yield the correct result ($5 \times 6 = 30$).

- **Test 3 (Large Random Multiplication):** We used large random integers to demonstrate that the homomorphic property holds true for realistic inputs, not just small constants.
- **Test 4 (Semantic Security):** We encrypted the same message twice to verify that the resulting ciphertexts are different, confirming the system is probabilistic and secure against pattern analysis.
- **Test 5 (Identity Element):** We tested multiplication by 1 to ensure the mathematical rule $x \cdot 1 = x$ is preserved in the encrypted domain.
- **Test 6 (Associativity):** We checked if $(A \cdot B) \cdot C$ equals $A \cdot (B \cdot C)$ to ensure that the order of operations does not affect the final decrypted result.
- **Test 7 (Commutativity):** We verified that $A \cdot B$ equals $B \cdot A$ to confirm that the order of operands is irrelevant, as expected in standard multiplication.
- **Test 8 (Boundary Condition):** We tested the maximum possible message size ($p-1$) to ensure the system handles boundary values without modular arithmetic errors.
- **Test 9 (Error Handling):** We attempted to encrypt a message larger than the modulus p to verify that the system correctly raises an exception instead of producing a silent calculation error.
- **Test 10 (Key Integrity):** We attempted decryption with an incorrect private key to prove that confidentiality is maintained and unauthorized keys produce incorrect results.

2. Argument for Test Coverage

We believe these test cases provide strong coverage because they balance standard operations with edge cases and security checks.

- **Breadth:** We covered the "happy path" (Tests 1-3), mathematical rules (Tests 5-7), and negative scenarios (Tests 9-10).
- **Focus:** We prioritized testing the homomorphic property and modular arithmetic boundaries, as these are the most complex parts of the implementation where bugs are most likely to occur.
- **Efficiency:** We avoided redundant tests (like testing 0 or adding more wrong keys) since the selected cases already cover the underlying logic sufficient to prove correctness.

Task 2: Application of partially homomorphic encryption: Paillier

```
=====
Paillier Homomorphic Encryption - Shopping Cart Demo
=====

Step 1: Client generates Paillier keypair
✓ Keys generated (key size: 2048 bits)

Step 2: Client populates shopping cart
Shopping cart (plaintext):
  Item 1: price=2000, quantity=1
  Item 2: price=120, quantity=5
  Item 3: price=1999, quantity=3

Expected total: 8597

Step 3: Client encrypts cart quantities
✓ Quantities encrypted (prices remain in plaintext)

Step 4: Server computes encrypted total cost
✓ Server computed encrypted total (without seeing quantities)

Step 5: Client decrypts the total cost
✓ Decrypted total: 8597

VERIFICATION:
  Expected total: 8597
  Decrypted total: 8597
  Match: ✓ YES
=====
PRIVACY DEMONSTRATION:
=====
What the server sees:
- Prices: [2000, 120, 1999] (plaintext)
- Quantities: [EncryptedNumber(...), EncryptedNumber(...), EncryptedNumber(...)]

What the server does NOT see:
- Actual quantity values: [1, 5, 3]
- Individual item costs: [2000, 600, 5997]

The server can compute the total cost without learning:
✓ How many of each item was purchased
✓ The cost of individual items in the cart
```

(a) Which operations does the Paillier encryption scheme support?

The Paillier encryption scheme is **additively homomorphic**. It specifically supports two main operations on encrypted data:

1. **Homomorphic Addition:** Adding two ciphertexts results in the encryption of the sum of the underlying plaintexts.

- Formula: $D(E(m_1) + E(m_2)) = m_1 + m_2$

2. **Scalar Multiplication:** Multiplying a ciphertext by a plaintext scalar (integer) results in the encryption of the product.

- Formula: $D(E(m) \cdot k) = m \cdot k$

In our shopping cart application, we used scalar multiplication to calculate **price * encrypted_quantity** and homomorphic addition to sum these values for the total cost.

(b) Which data types does it operate on?

Mathematically, the core Paillier algorithm operates on large **integers** (specifically within the residue class modulo n^2).

However, the **phe Python library** used in this exercise implements an encoding scheme that abstracts this limitation. It allows the scheme to operate on:

- **Integers (int)**
- **Floating-point numbers (float)**

The library automatically scales and encodes floating-point numbers into integers before encryption and decodes them back after decryption. In this specific task, we primarily worked with integers for the quantities and prices.

Task 3 : El Gamal Shopping Cart

```
=====
Task 3: ElGamal Homomorphic Encryption - Shopping Cart
=====

Step 1: Client generates ElGamal keypair
✓ Keys generated (Modulus p size: 256 bits)

Step 2: Client populates shopping cart
Shopping cart (plaintext):
  Item 1: Price=100, Quantity=2
  Item 2: Price=50, Quantity=4
  Item 3: Price=10, Quantity=5

Expected total: 450

Step 3: Client encrypts cart (Price AND Quantity)
✓ Price encrypted
✓ Quantity encrypted
(Both must be encrypted for ElGamal multiplication)

Step 4: Server computes encrypted subtotals
✓ Server calculated  $E(\text{Price} * \text{Quantity})$  for each item
! Note: Server CANNOT sum these (ElGamal is not additively homomorphic)

Step 5: Client decrypts and sums locally
[Client Internal Log] Decrypting received subtotals...
✓ Final Calculated Total: 450

=====
VERIFICATION:
Expected total: 450
Decrypted total: 450
Match: ✓ YES
=====
PRIVACY DEMONSTRATION (ElGamal Architecture):
=====
What the server sees:
- Prices: [Encrypted(...), Encrypted(...), Encrypted(...)]
- Quantities: [Encrypted(...), Encrypted(...), Encrypted(...)]

What the server does NOT see:
- Actual Prices: [100, 50, 10]
- Actual Quantities: [2, 4, 5]
- Total Sum: 450

Difference from Paillier:
✓ In Paillier, Server saw Plaintext Prices.
✓ In ElGamal, Server sees NOTHING (Full Privacy).
! Trade-off: Client has to do more work (Summation).
```

(a) Which changes did you have to make, and why?

We implemented two fundamental architectural changes compared to the Paillier implementation:

1. **Operands (Encryption of Price):** In the ElGamal implementation, the client must encrypt **both** the price and the quantity. In the Paillier version, only the quantity was encrypted. This change is necessary because ElGamal is multiplicatively homomorphic ($E(m_1) \times E(m_2) \Rightarrow E(m_1 \cdot m_2)$), requiring both operands to be ciphertexts to perform multiplication on the server.
2. **Server Responsibility (Aggregation):** The Server_EG class cannot compute the final total sum. Instead, it computes a **list of encrypted subtotals** (one for each item) and returns this list to the client. This is because ElGamal does not support homomorphic addition ($E(m_1) + E(m_2)$) not equal $E(m_1 + m_2)$, so the server cannot aggregate the results into a single ciphertext.

(b) What do your changes mean for the client (advantages/disadvantages)?

- **Disadvantage (Computational Load):** The client's workload increases significantly. Instead of decrypting a single "Total Cost" ciphertext, the client must decrypt N ciphertexts (where N is the number of items) and perform the summation locally.
- **Disadvantage (Bandwidth):** The communication overhead increases because the server transmits a list of ciphertexts back to the client instead of a single aggregated value.
- **Advantage (Privacy Model):** The privacy model is enhanced. Since the prices are also encrypted, the server computes the cost without seeing the individual item prices (unlike the Paillier implementation where prices were plaintext), ensuring full data confidentiality for the cart contents.

Task 4: Fully homomorphic encryption

```
=====
Task 4: Fully Homomorphic Encryption - Mean Computation
=====

PART 1: Integer Mean of 6 Encrypted Values

Compiling circuit...
✓ Compiled

Rationale: Basic arithmetic correctness
Test 1.1: Simple sequence
Input: [10 20 30 40 50 60]
Expected: 35
Result: 35
Status: ✓ PASS

Rationale: Homogeneous data, exact division
Test 1.2: All same values
Input: [5 5 5 5 5 5]
Expected: 5
Result: 5
Status: ✓ PASS

Rationale: Integer truncation (21/6 = 3.5 → 3)
Test 1.3: Sequential values
Input: [1 2 3 4 5 6]
Expected: 3
Result: 3
Status: ✓ PASS

Rationale: Upper bound test, no overflow
Test 1.4: Large values
Input: [100 200 300 400 500 600]
Expected: 350
Result: 350
Status: ✓ PASS

Rationale: Edge case with zero value
Test 1.5: Zero included
Input: [0 10 20 30 40 50]
Expected: 25
Result: 25
Status: ✓ PASS
```

```
PART 3: Mean of 7 Encrypted Values (Extension)

Compiling circuit for 7 values...
✓ Compiled

Rationale: Demonstrates adaptation to 7 values
Test 3.1: Seven values
Input: [10 20 30 40 50 60 70]
Expected: 40
Result: 40
Status: ✓ PASS

Rationale: Exact division by 7
Test 3.2: Seven same values
Input: [7 7 7 7 7 7 7]
Expected: 7
Result: 7
Status: ✓ PASS

=====
TEST COVERAGE SUMMARY:
Tests passed: 11/11

Coverage Dimensions:
✓ Arithmetic correctness (basic operations)
✓ Edge cases (zero, homogeneous data)
✓ Boundary values (small and large numbers)
✓ Precision handling (integer truncation, decimal scaling)
✓ Scalability (extension to 7 values)

=====

DEMONSTRATION: Separate Encrypt/Compute/Decrypt Steps

Input: [10 20 30 40 50 60]
✓ Encrypted
✓ Computed on encrypted data
✓ Decrypted: 35
Expected: 35
✓ CORRECT
```

```
PART 2: Mean with 2 Decimal Precision

Compiling circuit...
✓ Compiled

Rationale: Tests .50 precision (12.50)
Test 2.1: Decimal .50 result
Input: [10 11 12 13 14 15]
Expected: 12.50 (scaled: 1250)
Result: 12.50 (scaled: 1250)
Status: ✓ PASS

Rationale: Integer result in decimal form (21.00)
Test 2.2: Clean integer result
Input: [6 12 18 24 30 36]
Expected: 21.00 (scaled: 2100)
Result: 21.00 (scaled: 2100)
Status: ✓ PASS

Rationale: Truncates .333... to .50 (3.50)
Test 2.3: Repeating decimal
Input: [1 2 3 4 5 6]
Expected: 3.50 (scaled: 350)
Result: 3.50 (scaled: 350)
Status: ✓ PASS

Rationale: Baseline scaled precision (1.00)
Test 2.4: Homogeneous data
Input: [1 1 1 1 1 1]
Expected: 1.00 (scaled: 100)
Result: 1.00 (scaled: 100)
Status: ✓ PASS
```

```
=====
LIMITATIONS DEMONSTRATION:
=====

1. Fixed Input Size:
- Circuit compiled for 6 values cannot process 5 or 7
- Each size requires separate compilation

2. Integer Division Loss:
- Mean of [1,2,3,4,5,6] = 3 (not 3.5)
- Solution: Use scaled version (Part 2)

3. Limited Decimal Precision:
- Scaled version only provides 2 decimal places
- Mean of [1,2,3] = 3.50 (actual: 3.333...)

4. Input Range Restriction:
- Values must be within inputset range
- Testing 1000+ would fail without recompilation

5. Compilation Overhead:
- Each circuit compilation takes seconds
- Not suitable for dynamic applications
```

We originally tried to use the concrete-numpy library on Windows. However, we found that it is not compatible with Windows systems. Therefore, we used Google Colab (Linux environment) and installed the concrete-python library (the modern successor to concrete-numpy). The logic remains the same, but the library commands are updated.

(a) How can you obtain the mean of a list with seven encrypted integers?

We cannot use the same circuit for 7 integers because FHE circuits have a **fixed input size**. The circuit we compiled specifically expects 6 inputs.

To solve this, we did the following:

1. We defined a new function that sums 7 items and divides by 7.
 2. We updated the inputset to include arrays of size 7.
 3. We **compiled a new circuit** for this specific size. *(We demonstrated this successfully in "Part 3" of our code.)*
-

(b) What are the limitations of your implementations?

We identified three main limitations in our solution:

1. **Static Input Size:** The circuit is not dynamic. If we want to process a list of 5 or 8 numbers, we must compile a new circuit from scratch.
 2. **No Native Decimals:** The FHE library only works with integers. In our first version, we lost the decimal part of the mean (e.g., 3.5 became 3).
 3. **Limited Precision:** In our second version, we multiplied inputs by 100. This gives us exactly **two decimal digits** of precision. Any detail beyond that is lost.
-

(c) Provide the rationale for your test case selection.

We designed a comprehensive suite of tests categorized into four specific groups:

1. Baseline & Arithmetic Correctness

- **Tests used:** **Test 1.1** and **Test 2.2**.
- **Rationale:** These tests resulted in clean integer division. We used them to verify that the fundamental encryption, addition, and division logic perform correctly under standard conditions.

2. Precision & Truncation Logic (Crucial Test)

- **Tests used:** **Test 1.3** vs **Test 2.3**.
- **Rationale:** This comparison validates the core requirement.
 - **Test 1.3** confirms that the integer implementation correctly truncates the result (3.5 -> 3).
 - **Test 2.3** proves that our scaling strategy preserves the precision (3.5 -> 3.50), effectively solving the integer limitation.

3. Boundary Analysis (Limits)

- **Tests used:** **Test 1.4** (Upper Bound).
- **Rationale:** We tested values up to 600 to ensure the circuit parameters were sufficient to prevent overflow errors with larger inputs.

4. Edge Cases

- **Tests used:** **Test 1.5** (Zero value) and **Test 1.2** (Homogeneous data).
- **Rationale:** We tested lists containing zero and identical values to ensure the division logic introduces no noise when variance is zero or inputs are minimal.

Argument for Test Coverage:

By covering the operational range from zero (Test 1.5) to large inputs (Test 1.4), and explicitly comparing truncated vs. scaled results (Test 1.3 vs 2.3), we ensured the solution is robust for all expected scenarios.